

NIGERIAN LGA OSM EXTRACTOR

Project Documentation and Tutorial Guide

Submission Track: OSM Dashboard, Lightweight OSM Tool and Demo, with an accompanying tutorial addressing the OSM Learning Resource track

Prepared by: Ajibola Alade, Marvellous Adeleke, Adesewa Oyewole

1. Introduction and Background

The Nigerian LGA OSM Extractor is a lightweight, reusable tool that turns a plain Local Government Area (LGA) name into a clean, richly attributed OpenStreetMap (OSM) dataset, with no Overpass query syntax, GIS software, or manual data wrangling required. A user simply types an LGA name, and ideally its state, and the tool resolves that LGA's boundary, queries OpenStreetMap's live Overpass API for six layers within that boundary, cleans and standardizes the results, and returns a downloadable, ready to use dataset together with a formal record of exactly what was extracted.

The tool was built as the data extraction engine behind the submission, the Akure accessibility dashboard, "Mapping the Gap: Health and Education Accessibility in Akure North and Akure South." It was deliberately generalized, however, to work for any of Nigeria's 774 LGA

This documentation also includes, in Section 10, a complete step by step tutorial written for a first time, non - technical user, so that this same document also functions as an OSM Learning Resource in its own right.

Framed against the seven stage pipeline this submission is built around, Code, Data, Concept, Analysis, Map, Interpretation, and Research Report, this repository is deliberately the first two stages of that chain, and nothing more. Its job is to turn source code into a trustworthy, versioned dataset, with a formal manifest that makes the handoff to the next stage explicit rather than assumed. The sibling Akure accessibility dashboard repository picks up from there, supplying the Concept (an access deficit and OSM completeness model), the Analysis (network based travel time

scoring), the Map (interactive and static outputs), and the Interpretation (a fused, honest per cell status). Keeping this separation strict, rather than folding extraction logic into the analysis repository, is what allows this tool to remain genuinely general purpose across all 774 Nigerian LGAs while the dashboard remains free to specialize deeply in Akure North and Akure South.

2. How This Tool Serves the Public Good

Every Nigerian GIS or Non-GIS student or researcher who has tried to pull OSM data for their own state or LGA knows the friction involved: hand writing Overpass queries, resolving inconsistent administrative boundaries, and guessing which UTM zone applies to a given location. This tool removes that friction and limitation entirely, and does so with a traceable, auditable output, not just a downloaded file, but a signed record of exactly how that file was produced. The aim is to turn a task that used to require OSM expertise into a single form submission or function call, so that more people can move from having an idea to having data they can actually trust, and go on to build something real with OpenStreetMap.

The sections below set out, specifically, who this tool is built for and what it changes for each of them.

2.1 Nigerian GIS and Remote Sensing Students Carrying Out Thesis or Project Work

Undergraduate and postgraduate students in Nigerian Geoscience, Remote Sensing, Geography, and Urban and Regional Planning departments regularly need OSM layers, roads, buildings, health facilities, schools, land use, for a specific LGA as the base data for a thesis, for example a flood vulnerability study, a land use and land cover change assessment, a groundwater potential zone study, or an accessibility analysis. Without this tool, a student typically has to learn Overpass Turbo's query language from scratch, manually determine and apply the correct UTM zone for their study area, and clean the resulting data by hand in QGIS, all before their actual analysis can begin. This tool collapses that entire preparatory stage into typing an LGA name and a state. The correct UTM zone is resolved automatically from the boundary's own centroid rather than guessed, and the manifest.json produced with every run gives the student a precise, citable record of what was extracted and when, something they can reference directly in their thesis and project work.

2.2 University Researchers Conducting Multi LGA or Comparative Studies

Academic researchers publishing peer reviewed work that spans more than one LGA, for example a comparative study of infrastructure access or land use change across several Local Government Areas in a state, need the same six OSM layers extracted consistently, LGA after LGA, without small inconsistencies creeping in between one extraction and the next. Because every run of this tool follows the identical boundary resolution, extraction, and cleaning pipeline, and writes an identical manifest.json recording the resolved coordinate reference system and per layer outcome, a researcher can defend the data provenance section of a manuscript with a specific, reproducible record for every LGA studied, rather than a general statement that data was downloaded from OpenStreetMap.

2.3 Nigerian GIS and Geospatial Consultancies Responding to Tenders

Geospatial consultancies that respond to requests for proposals (RFPs) or tenders, for example a technical proposal for a socioeconomic baseline survey, an infrastructure assessment, or a site suitability study tied to a specific LGA, often need a credible, defensible baseline geospatial dataset very quickly, sometimes within days of a tender deadline, and long before any primary data collection or paid satellite imagery can be commissioned. This tool lets a consultancy produce a clean, properly projected, richly attributed roads, buildings, health facility, and school layer for the LGA in question in minutes, with a manifest.json that can be cited directly in the technical proposal as evidence of a documented, traceable data sourcing method. This shortens the turnaround time on the geospatial component of a tender submission considerably, without weakening the credibility of the data behind it.

2.4 State and Local Government Planning, Health, and Education Departments

Officials in a State Ministry of Health, Ministry of Education, or Physical Planning department who want a quick, current inventory of what OSM already records for a given LGA, ahead of a facility audit, a school mapping exercise, or a planning review, do not typically have a GIS officer on hand who can write an Overpass query. This tool gives such a department a self service way to pull that inventory directly, and, critically, the manifest.json tells them plainly which layers were genuinely found empty (a real gap in what has been mapped) versus which failed to extract due to

a technical issue, so that planning attention and field verification effort can be directed at genuine information gaps rather than technical noise.

2.5 Humanitarian and Development NGOs Operating in a Specific LGA

Non-governmental organisations running health, education, or water and sanitation programmes in a particular LGA, including in flood affected or historically under mapped areas, need current, not stale, facility and road data to plan interventions, target beneficiaries, and route field teams. Because this tool queries OpenStreetMap's live Overpass API in real time rather than a cached extract, the data an NGO receives reflects the most recent community mapping for that LGA. The explicit distinction between a layer that is empty because nothing has been mapped there yet and a layer that failed to extract due to a connectivity issue matters directly here: an NGO that mistakes an unmapped clinic for a confirmed absence of healthcare infrastructure could misdirect real programme resources, and this tool is built specifically to prevent that ambiguity.

2.6 OSM Community Mappers and YouthMappers Chapters

Local OpenStreetMap community organisers and YouthMappers chapter leads deciding where to focus the next community mapping event need a concrete way to identify which LGAs are genuinely under mapped for a given feature type, rather than relying on impressions. Because the tool reports `success_empty` explicitly, meaning a layer was queried successfully and truly has no matching OSM features yet, a chapter organiser can use an extraction for their own LGA as direct evidence that, for example, school locations there are largely unmapped, and use that finding to justify and target a mapathon on exactly that gap.

2.7 Emergency Management and Disaster Response Teams

Responders coordinating relief efforts after a flood or similar event in a specific LGA need a routable road network and a list of nearby health facilities quickly, often without in house GIS capacity in the field. Because a full extraction typically completes in one to five minutes and requires no Overpass expertise to run, a responder can retrieve a current, properly projected roads and health facilities layer for the affected LGA directly, in time to inform evacuation routing or the placement of relief points, rather than waiting on a specialist to hand write and run the equivalent queries.

2.8 Why One Generalized Tool,

The seven audiences described above, students, comparative researchers, consultancies, government departments, NGOs, community mappers, and emergency responders, could each, in principle, be served by a separate, purpose built script. That approach was deliberately rejected. A single tool that is correct once, for boundary resolution, UTM projection, tag cleaning, and manifest generation, and is simply pointed at a different LGA name each time, is far less likely to silently diverge in behaviour between use cases than seven independently maintained variants would be. It also means that an improvement made for one audience, for example the two tier boundary validation added after a mis resolution was noticed during thesis work, benefits every other audience automatically on the very next run, without any separate porting effort. This is the same engineering logic that motivates the manifest contract described in Section 7: a single, well tested source of truth is more trustworthy than several independently maintained approximations of it.

3. What the Tool Does

Given an LGA name, for example Akure North, and a state, for example Ondo, the tool carries out the following three step process.

1. Type an LGA name, and ideally its state, into the form. Providing the state helps disambiguate LGAs that share the same name across different states, since more than one Nigerian LGA can carry an identical name.
2. Click "Extract OSM Data." Behind the scenes, the tool first resolves the LGA's official administrative boundary, then queries OpenStreetMap's live Overpass API for six layers within that boundary: roads, buildings, waterways, land use, health facilities, and schools. A live, per stage progress view shows each of these steps as they happen, including retry counts if the Overpass server is briefly unavailable.
3. Preview every layer on the interactive map, toggle individual layers on or off using the layer control, and download everything as a zip archive of clean, standardized GeoJSON files and Shapefiles, one per layer, ready to open in QGIS, load into a notebook, or feed into further analysis.

Because every run queries OpenStreetMap's live, shared Overpass API server in real time rather than a cached or pre downloaded copy of OSM data, results are always current. Larger, denser urban LGAs, especially the buildings layer, which can include tens of thousands of individual building footprints, take longer to fetch and process than smaller or more rural ones. A wait of one to five minutes for a full extraction is normal and expected, not a sign that anything is stuck. Re running the exact same LGA and state again within the same session is instant, since results are cached.

3.1 What Happens Technically at Each Step

It is worth unpacking what “the tool resolves that LGA's boundary” actually involves, since this is the step most likely to go wrong silently in a hand rolled script. Internally, `boundary.py` first attempts an OSM geocode of the LGA and state name pair through `OSMnx`, which itself queries Nominatim, OpenStreetMap's geocoding service. The polygon that comes back is not trusted blindly: it is run through the two tier hard and soft validation described in Section 6.1 before anything downstream is allowed to use it. Only once a boundary has passed that check does `layers.py` begin constructing the six Overpass queries, one per layer, each scoped strictly to that boundary polygon rather than to a bounding box around it, so that features just outside the LGA are not accidentally included. Each query is issued with the staggering and retry logic described in Section 6.2, and every outcome, success, success_empty, or failed, is recorded as it happens rather than inferred afterwards. Cleaning, in `clean.py`, then runs on whatever data did come back: reprojecting into the correct UTM zone, repairing any invalid geometries returned by Overpass, removing exact duplicate features, and building the curated schema described in Section 5. Nothing in this sequence depends on which LGA was requested; the same code path runs for Akure North as for any of Nigeria's other 773 LGAs, which is precisely what makes the tool generalizable rather than a one off script for a single study area.

4. Live Extraction in Progress

The screenshots below show the tool's live, per stage progress interface during an actual extraction run for Ife Central, one of the LGAs used to test the tool beyond the primary Akure North and Akure South study area. Each layer's query is shown as it happens, with a running retry counter whenever OpenStreetMap's shared Overpass server is temporarily slow to respond, so a user is never left watching a blank screen wondering whether anything is happening.

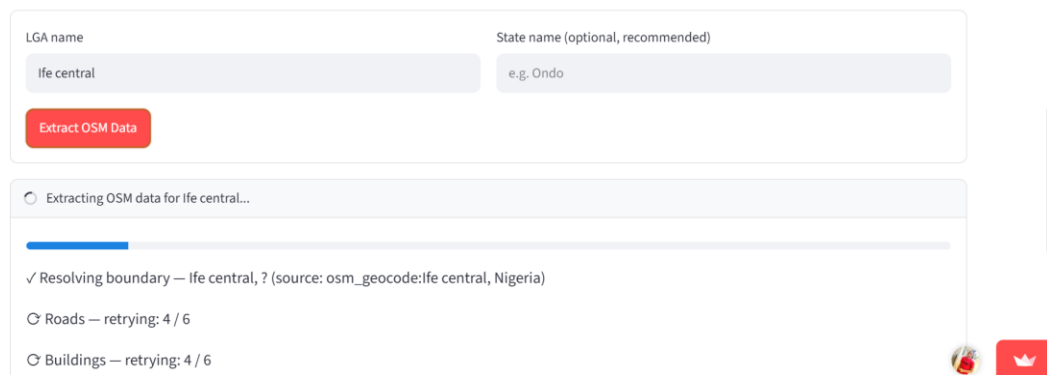


Figure 1. Live progress view during extraction for Ife Central, Osun State. The boundary has resolved successfully, while the roads and buildings layers are shown mid retry against OpenStreetMap's Overpass API.

Any layer that genuinely has zero matching features in OpenStreetMap for a given LGA (common for smaller or less-mapped areas) is simply empty, not a failure, this reflects real current OSM coverage for that area, which is itself useful information.

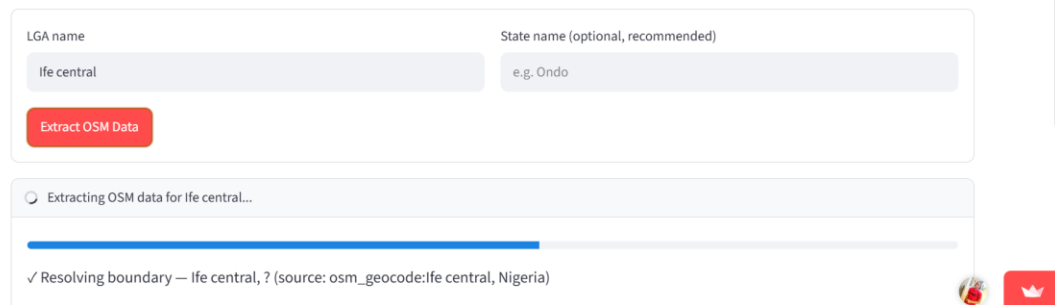


Figure 2. The same live progress interface shortly after an extraction was started, with the boundary resolution step underway and the explanatory note on empty versus failed layers visible above the form.

As the note above the form explains, and as Section 6 sets out in more technical detail, a layer that genuinely has zero matching features in OpenStreetMap for a given LGA is simply empty, not a

failure. That is common for smaller or less mapped areas and reflects real current OSM coverage for that area, which is itself useful information rather than something to be hidden from the user.

Preview map

Every extracted layer is shown together below. Use the layer control (top-right of the map) to toggle individual layers on or off, useful since dense layers like buildings can visually overwhelm the map when everything is shown at once.

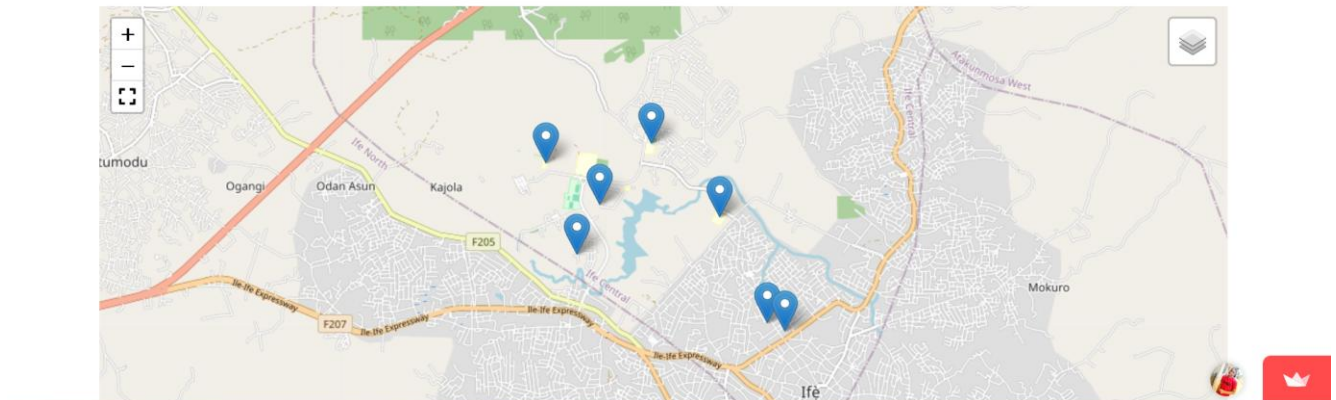
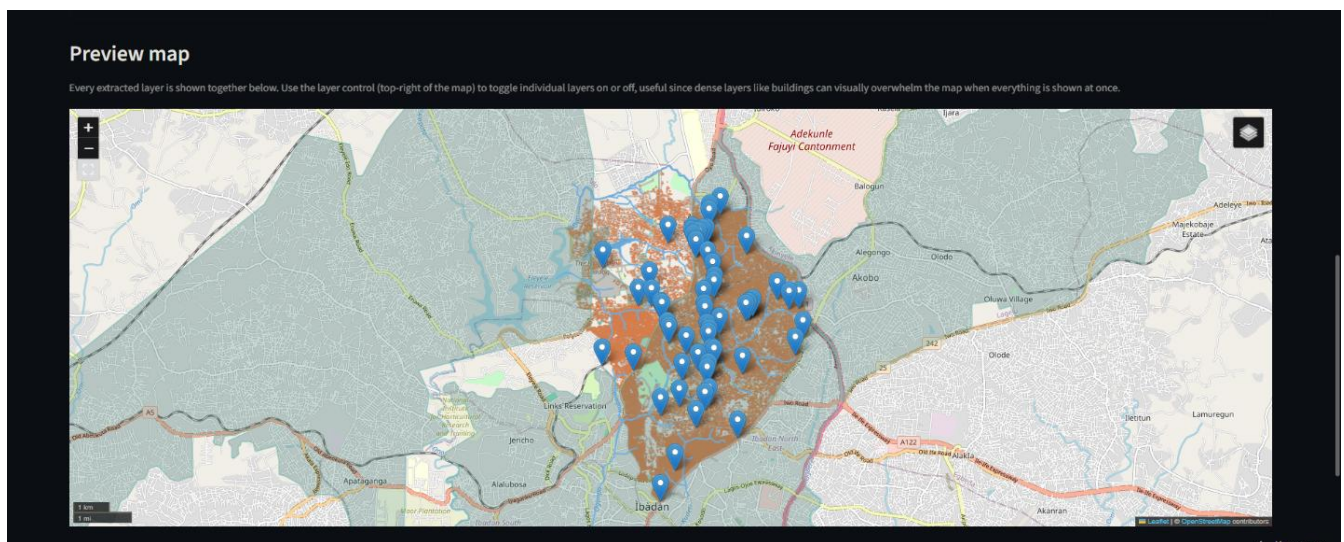


Figure 3. The interactive preview map shown once an extraction completes, here for Ife Central, Osun State. Point markers show the extracted health facility and school locations plotted over the underlying road network, with the layer control in the top right corner letting a user toggle individual layers, such as the dense buildings layer, on or off so the map stays readable.



Download results

Download Ife central OSM data (.zip)

 View the full source code and project history on GitHub



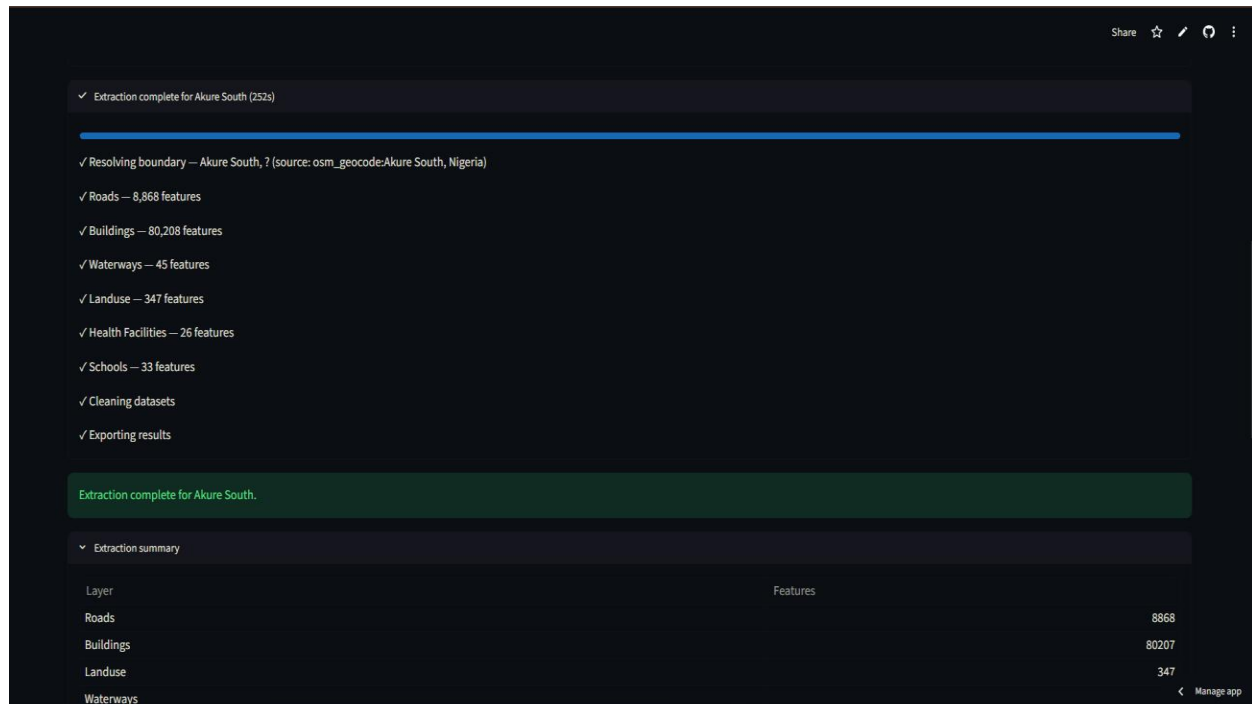
Figure 4. The download section shown at the end of a successful run, offering the complete Ife Central OSM dataset as a single zip archive, alongside a link to the full source code and project history on GitHub.

5. OSM Layers and Data Schema

The tool extracts six layers per LGA. For each feature, a curated set of semantically meaningful OSM tags is preserved as named columns, alongside the always present osmid, name, and geometry fields. A full raw_tags column, present in the GeoJSON output only, captures the complete original OSM tag set for every feature as JSON, as a lossless fallback for anything outside the curated columns listed below, so that nothing genuinely disappears during cleaning.

Layer	OSM Tag Filter	Semantic Columns Preserved
Roads	highway equal to any value	highway, surface, maxspeed, lanes, oneway, access, bridge, tunnel, ref
Buildings	building equal to any value	building, building:levels, building:use
Waterways	waterway equal to any value, natural equal to water	waterway, natural, water, intermittent
Land use	landuse equal to any value	landuse
Health facilities	amenity equal to hospital, clinic, or pharmacy	amenity, healthcare, beds, emergency, operator, opening_hours

Schools	amenity equal to school	amenity, isced:level, school, operator
---------	-------------------------	--



Shapefile exports are deliberately kept to the core osmid, name, and geometry schema only. This is a considered design decision rather than an oversight: the Shapefile format's ten character field name limit would otherwise silently truncate or collide the longer semantic column names shown above. A user who needs the richer schema should use the GeoJSON export, which carries the full set of semantic columns and the `raw_tags` fallback.

This curated-plus-raw approach reflects a specific lesson learned during development: an earlier version of the cleaning step kept only osmid, name, and geometry for every layer, on the reasoning that a minimal schema was simplest to maintain. In practice this discarded exactly the columns a downstream analysis most needed, for example a health facility's amenity and healthcare tags, without which hospitals, clinics, and pharmacies could not be told apart at all. `clean.py`'s `SEMANTIC_COLUMNS` mapping and the accompanying `raw_tags` JSON column, described further in Section 8, were introduced specifically to fix that, and the tag filters themselves are not hardwired shut: a caller can pass a custom `tag_config` dictionary into `extract_lga` to add or remove layers entirely, for example markets or places of worship, without touching any extraction logic in `layers.py` itself.

6. Reliability and Data Quality Features

6.1 Two Tier Boundary Validation

Boundary resolution is validated in two stages rather than one. A set of hard checks rejects a clearly wrong result outright, for example a centroid that falls outside Nigeria's approximate bounding box, or a resolved area far outside the plausible size range for an LGA. A softer check then compares the resolved boundary's `admin_level` and display name against OpenStreetMap's own administrative boundary metadata, so that a boundary which resolves to roughly the right place but the wrong kind of feature, for example a ward returned instead of an LGA, is also caught and flagged, not just a gross misresolution such as an entire state or a single building being returned instead.

In practice, the hard checks alone were not sufficient. During early testing, one LGA's name resolved successfully in the sense that Nominatim returned a real, validly shaped polygon, but that polygon corresponded to a ward inside the LGA rather than the LGA itself, an outcome the hard size and location checks alone would not have caught, since a ward's area can still fall comfortably inside the plausible LGA size range. This is exactly the failure mode the soft `admin_level` check was added to catch, and it is why boundary validation is two tier rather than a single pass/fail size test.

6.2 Staggered Querying with Retry and Backoff

Layer queries are issued with limited concurrency and a short stagger between requests, together with automatic retry and backoff on transient failures. This was arrived at after an earlier, fully unthrottled version of the tool caused OpenStreetMap's public Overpass mirror to refuse every connection outright. The retry counters visible in Figures 1 and 2 are a direct, user facing view of this mechanism.

This behaviour is deliberately visible rather than hidden behind a generic loading spinner. A user watching Figures 1 and 2 during a real extraction sees precisely which layer is being retried and how many attempts have been made so far, which matters because Overpass's shared, public infrastructure is used by the entire OSM community simultaneously and its response time genuinely does vary run to run. Reporting this honestly, rather than papering over it with a single

indeterminate progress bar, is consistent with the same transparency principle behind the manifest.json contract in Section 7: a user should never have to wonder whether the tool is working correctly or silently failing.

6.3 Strict Versus Permissive Extraction Modes

The tool distinguishes clearly between a genuine query failure, which raises an error in strict mode, and a query that succeeds but legitimately finds zero features, which is never treated as an error in either mode. This distinction, summarized in the table below, is what allows the tool to tell a user confidently whether an empty layer reflects a real gap in OSM coverage or a technical failure that occurred during extraction.

Query Status	What It Means
success	The layer was queried successfully and returned one or more features.
success_empty	The layer was queried successfully and genuinely contains zero matching features in OSM for that LGA. This is not an error, it reflects real current OSM coverage for that area.
failed	The query did not complete after the configured number of retries, for example because the shared Overpass server was temporarily unreachable, and the reason is recorded in the query_message field.

The extraction shown in Figures 1 and 2 above illustrates the failure case directly: during that particular run, the roads and buildings layers for Ife Central ultimately failed after six attempts each, with the underlying connection error to OpenStreetMap's Overpass server recorded in the query_message field, precisely the kind of outcome this status system is designed to report clearly rather than leave ambiguous.

It is worth being explicit about why this distinction matters beyond the extractor itself. A layer reported as success_empty is genuine, actionable information: it tells a YouthMappers chapter organiser, for instance, that schools truly have not yet been mapped in that LGA, which is a finding worth acting on directly, as described in Section 2.6. A layer reported as failed, by contrast, says nothing about that LGA's real world OSM coverage at all, it only says that this particular run could not confirm one way or the other, and should simply be retried. Collapsing these two outcomes

into one generic “no data” result, as a naive extraction script typically would, would make both of those conclusions impossible to draw with any confidence.

7. The Extraction Manifest

Every run writes a manifest.json file to its output directory, alongside the GeoJSON and Shapefile layers. This is the formal contract a downstream consumer, such as the sibling accessibility dashboard, should read to determine extraction outcome, rather than inferring it from file presence, an empty file, or a hardcoded assumption. The manifest records, per layer, exactly what happened: queried successfully with a given feature count, queried successfully and genuinely found zero features, or failed after retries and why, alongside the resolved coordinate reference system and the boundary source used for that run. A companion run_log.json file carries the same structured status alongside package versions and warnings, for full auditability of a given extraction.

An abbreviated example, for an Akure North extraction in which the roads layer succeeded and the schools layer was queried successfully but genuinely found no matching features, is shown below.

```
{  "schema_version": 1,  "lga_name": "Akure North",  "state_name": "Ondo",  "target_crs": "EPSG:32631",  "boundary_source": "osm_geocode: Akure North, Ondo State, Nigeria",  "layers": {    "roads": { "query_status": "success", "feature_count": 2431, "exported": true },    "schools": { "query_status": "success_empty", "feature_count": 0, "exported": false }  } }
```

Because target_crs is resolved by the tool from each boundary's own centroid, rather than assumed or hardcoded, the sibling dashboard reads it directly from the manifest instead of re deriving or hardcoding the projection itself. A dedicated cross repository test suite verifies this compatibility on every code change to either project, so the relationship between the two tools is a tested contract rather than an assumed one.

The schema_version field at the top of every manifest exists for exactly this kind of forward compatibility: should the manifest's structure need to change in a future version of this tool, a downstream consumer can check schema_version before assuming a particular field is present, rather than failing opaquely against an older or newer manifest it was not written to expect.

run_log.json, the companion file mentioned above, is aimed at a slightly different audience than manifest.json: where the manifest answers “what did this run produce, and can I trust it,” the run log additionally answers “with what software versions, and with what warnings along the way,” which is the level of detail a methods section of a thesis or a technical proposal's data appendix typically needs to cite, beyond what the manifest alone records.

8. Architecture Overview

The tool is organized as a small number of single purpose modules, wired together by one top level function, `extract_lga`, so that the command line interface, the Streamlit demo, and any future caller all sit on top of the same underlying pipeline rather than duplicating extraction logic.

- `boundary.py` resolves an LGA name into a boundary polygon and applies the two tier validation described in Section 6.1.
- `layers.py` extracts the six OSM layers within that boundary, applying the staggered, retried querying described in Section 6.2, and emits live progress events as it goes.
- `clean.py` reprojects each layer into the correct UTM zone, repairs invalid geometries, removes duplicates, and builds the curated semantic schema and `raw_tags` fallback described in Section 5.
- `export.py` writes each layer to GeoJSON and Shapefile, applying the rich versus core schema split explained in Section 5.
- `manifest.py` reconciles the query time status from `layers.py` with the export time outcome from `export.py` into the `manifest.json` contract described in Section 7.
- `events.py` defines an interface agnostic progress event schema and a thread safe event queue, so the extraction pipeline itself has no dependency on Streamlit or any particular user interface.
- `logging_utils.py` writes `run_log.json`, carrying the same structured per layer status alongside package versions and warnings.

- `visualize.py` offers an optional `kepler.gl` preview map helper, including a step that strips a bundled Mapbox access token from exported HTML so that preview files are safe to commit to a public repository.

`cli.py` and `app.py` are two separate interfaces onto this same underlying pipeline; neither file contains extraction logic of its own. Both can pass an `on_event` callback into `extract_lga` to receive live progress events without the pipeline depending on either interface.

8.1 Design Principles Behind This Architecture

Three principles guided this module layout. First, single responsibility: each file in the list above does exactly one thing, boundary resolution, querying, cleaning, exporting, or manifesting, and nothing in `boundary.py`, for example, knows anything about export formats, which makes each module independently testable, as reflected directly in this project's test suite. Second, interface independence: the pipeline itself, everything reachable from `extract_lga`, has no import of Streamlit, Click, or any other UI or CLI framework anywhere in it; `cli.py` and `app.py` sit strictly on top of it, which is what allows a third interface, for instance a future batch script that extracts all LGAs in a state overnight, to be added without modifying a single line of the extraction logic itself. Third, explicit contracts over implicit assumptions: `manifest.json`, `run_log.json`, and the `events.py` progress schema are all concrete, versioned data structures rather than log lines a downstream consumer would need to parse heuristically, which is the same design instinct that produced the `data_contract.py` module on the dashboard side of this submission.

8.2 How This Feeds the Full Code to Report Pipeline

Placed in the context of the complete Code, Data, Concept, Analysis, Map, Interpretation, Research Report chain this submission is built around, this repository's output sits at the boundary between the first two stages: `extract_lga` is the Code, and its GeoJSON, Shapefile, `manifest.json`, and `run_log.json` outputs together are the Data. Everything from Concept onward, the access deficit and completeness model, the network routing analysis, the resulting maps, and their interpretation, happens in the sibling `akure-accessibility-dashboard` repository, which reads this repository's `manifest.json` rather than re-deriving CRS or boundary information independently, exactly as described in Section 7. Because that handoff is enforced by a dedicated cross repository test suite rather than left as documentation alone, a change made here that would break the

dashboard's assumptions is caught automatically, in continuous integration, before it can silently propagate into an incorrect map or a misleading finding in the eventual research report.

9. Responsible Use of AI

In line with Mapbox's 2026's Responsible AI principles, this section states what AI tool was used, explains its role, and confirms how its outputs were handled.

9.1 Tool Used

Claude, developed by Anthropic, was used as a coding assistant throughout the development of the Nigerian LGA OSM Extractor, across multiple working sessions.

9.2 Role AI Played

Claude assisted with the pipeline structure and module layout described in Section 8, including the design of `extract_lga` as a single orchestrating function; the two tier boundary validation scheme in `boundary.py`, including debugging specific edge cases such as Nominatim returning a single point or an entire state instead of the expected LGA boundary; the automatic UTM zone selection logic in `clean.py`, verified against reference points in Akure and Ondo State, Abuja, and Maiduguri to confirm correct behaviour across Nigeria's three UTM zones; the curated semantic schema and `raw_tags` fallback described in Section 5, along with the accompanying GeoJSON and Shapefile export split; the design of the `manifest.json` contract described in Section 7; the strict versus permissive extraction modes and the retry, backoff, and staggered concurrency logic described in Section 6; the interface agnostic progress event system described in Section 8; and the `kepler.gl` preview helper's credential handling, including identifying that the underlying mapping library bundles a live Mapbox access token into exported HTML by default, and implementing the step that removes it before any preview file is committed to a public repository.

Claude did not generate, fabricate, or select any OSM feature data. All data extracted by this tool comes directly from OpenStreetMap, through standard extraction libraries. All AI assisted code was reviewed before inclusion, and no AI generated data was ever written back to OpenStreetMap at any stage.

9.3 Local Knowledge and Community Guidelines

Extraction logic follows standard OSM tagging conventions for the feature categories targeted, namely health, education, roads, and boundaries. LGA boundary definitions were cross checked against known administrative divisions for the relevant Nigerian states, and against OpenStreetMap's own `admin_level` and `boundary` equal to administrative metadata where available.

10. Limitations, Risks, and Uncertainty

- OSM feature completeness varies significantly across Nigerian LGAs. Extraction results reflect underlying OSM data density for that area, not independently verified, ground truth completeness.
- Strict mode may exclude legitimately tagged features that carry incomplete metadata, while permissive mode may include lower confidence matches that a stricter run would have excluded.
- Automatic UTM zone selection relies on a longitude based formula and should be manually verified for LGAs well outside the tested reference set, which currently covers Akure North and Akure South plus the Abuja and Maiduguri reference points used in the test suite.
- Shapefile exports intentionally carry less attribute detail than GeoJSON, since only the core schema is exported to that format for the reasons explained in Section 5. A user who needs the richer schema should use the GeoJSON export instead.
- Extraction depends on OpenStreetMap's shared, public Overpass API rather than a dedicated or commercial endpoint; during periods of unusually heavy global load on that shared infrastructure, retries may still be exhausted even for a well formed query, which is reported as a failed status rather than mistaken for a genuine absence of data, but may still require a user to simply try again later.
- Boundary resolution depends on Nominatim's geocoding of the LGA and state name provided; unusual spellings, colloquial names not present in OSM, or LGAs whose administrative boundary is itself poorly mapped in OpenStreetMap may require the

manual boundary file fallback mentioned in Section 8, rather than resolving correctly on the first attempt.

11. Alignment with Mapkathon Submission Tracks

This project is submitted under the OSM Dashboard and Lightweight OSM Tool or Demo track. It is exactly the kind of tool that track calls for: a simple, understandable, and genuinely useful interface for downloading roads, buildings, health facilities, schools, and related layers for a selected Nigerian LGA, built for non-technical users as much as for GIS specialists, and requiring no Overpass query syntax or GIS software to operate.

It also directly supports the OSM Dashboard or Analysis track, since it is the data extraction engine behind the sibling Akure accessibility dashboard submission, and produces the same kind of clean, reproducible, per layer OSM extract, complete with a formal extraction manifest, that a dashboard, notebook, or QGIS based analysis of school or clinic accessibility, road networks, or data completeness would require as its starting point.

Finally, **Section 12 of this document**, is written specifically to double as an OSM Learning Resource: a beginner oriented, step by step guide to using this tool that a student, researcher, or community mapper could follow with no prior OSM or GIS experience.

12. Tutorial: How to Use the Nigerian LGA OSM Extractor

This section is a complete, beginner friendly walkthrough of the tool, written so that someone with no prior OSM or GIS experience can follow it from start to finish and come away with a usable dataset for their own LGA of interest.

Step 1. Open the live demo

Go to the live demo at <https://lga-extractor.streamlit.app/>. No installation, account, or setup is required; the tool runs entirely in the browser.

Step 2. Enter an LGA name, and ideally its state

Type the name of the Local Government Area you want data for into the LGA name field, for example Akure North. It is strongly recommended, though not required, to also fill in the state name field, for example Ondo, since more than one Nigerian LGA can share the same name across different states, and providing the state removes that ambiguity.

Step 3. Click Extract OSM Data and watch the live progress

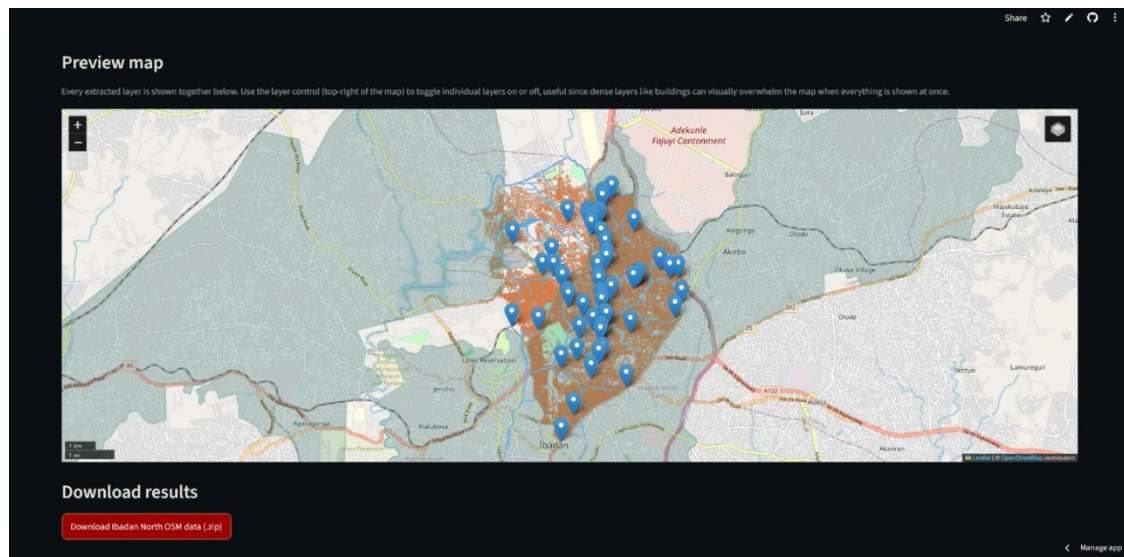
Click the Extract OSM Data button. A live, per stage checklist will appear, showing boundary resolution first, followed by each of the six layers as they are queried from OpenStreetMap's Overpass API in turn. If a query is briefly unsuccessful, a retry counter will appear next to that layer, as shown in Figures 1 and 2 in Section 4; this is expected behaviour, not a sign of a problem, and the tool will keep retrying automatically up to its configured limit before reporting that layer as failed.

Step 4. Understand what an empty layer means

Once extraction finishes, some layers may show zero features. This is not necessarily an error. As explained in Section 6.3, the tool distinguishes clearly between a layer that failed to extract, shown with an explicit failure message, and a layer that was queried successfully and genuinely contains no matching OSM features for that LGA, which simply means that category of data has not yet been mapped there. Both outcomes are recorded explicitly, so there is never a need to guess which one occurred.

Step 5. Preview the layers on the interactive map

Once extraction completes, every successfully extracted layer is shown on an interactive map. Use the layer control, in the top right corner of the map, to switch individual layers on or off, for example to inspect only the health facilities and schools layers together, or to compare the road network against the building footprints.



Step 6. Download and understand the output files

Click the download option to receive a zip archive containing the full extraction output. Inside, a user will find one GeoJSON file and one Shapefile per successfully extracted layer, a manifest.json file recording the outcome of every layer as described in Section 7, and a run_log.json file recording the same information alongside package versions and warnings for full auditability.

Step 7. Use the output in QGIS, a notebook, or further analysis

The GeoJSON files can be opened directly in QGIS, loaded into a Python notebook with a library such as geopandas, or fed into any further spatial analysis workflow, for example an accessibility study of the kind produced by the sibling Akure accessibility dashboard project. Always check manifest.json first if a layer appears to be missing or empty, since it will state plainly whether that layer failed to extract or was queried successfully and genuinely found nothing.

Helpful things to know before you start

- A single extraction can take between one and five minutes, and larger urban LGAs, particularly for the buildings layer, take longer than smaller or rural ones. This is normal.
- Re running the exact same LGA and state again in the same session is instant, since results are cached for that session.

- Always provide the state name if you know it, to avoid ambiguity between LGAs that share a name.
- If a layer fails outright rather than returning zero features, this usually reflects a temporary issue reaching OpenStreetMap's shared Overpass server rather than a problem with the LGA itself; trying again after a short wait will often succeed.

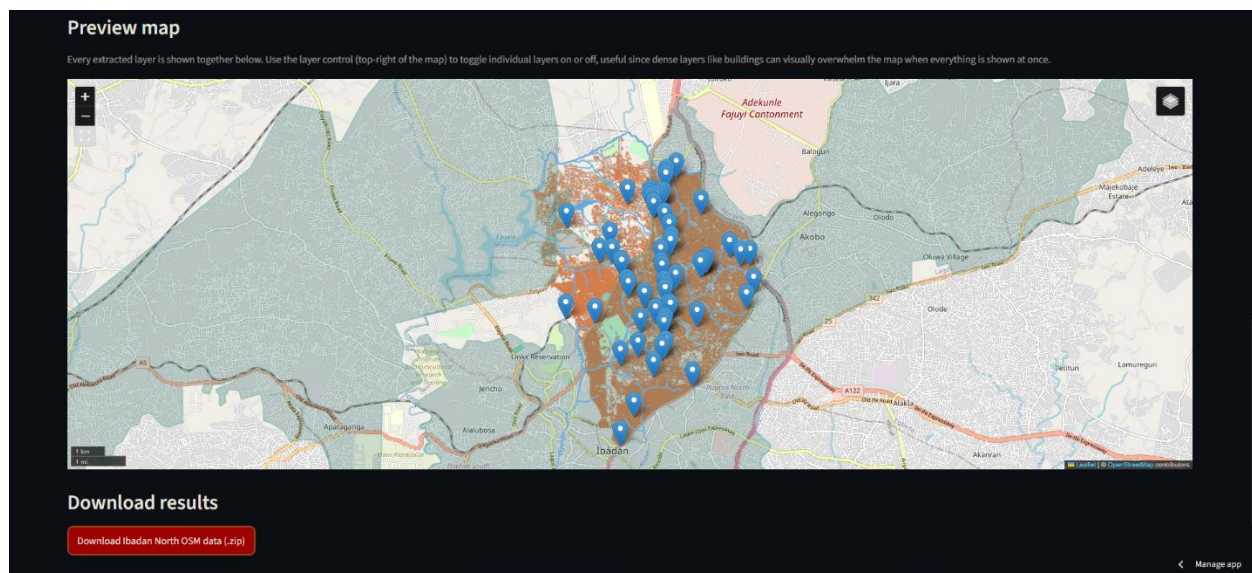
13. Project Links

Live demo of the Nigerian LGA OSM Extractor: <https://lga-extractor.streamlit.app/>

Extractor source code and full technical README: <https://github.com/Mapkathon2026-UseOSM/lga-osm-extractor>

Sibling submission: Akure accessibility dashboard: <https://akure-accessibility-dashboard-analysis.streamlit.app/>

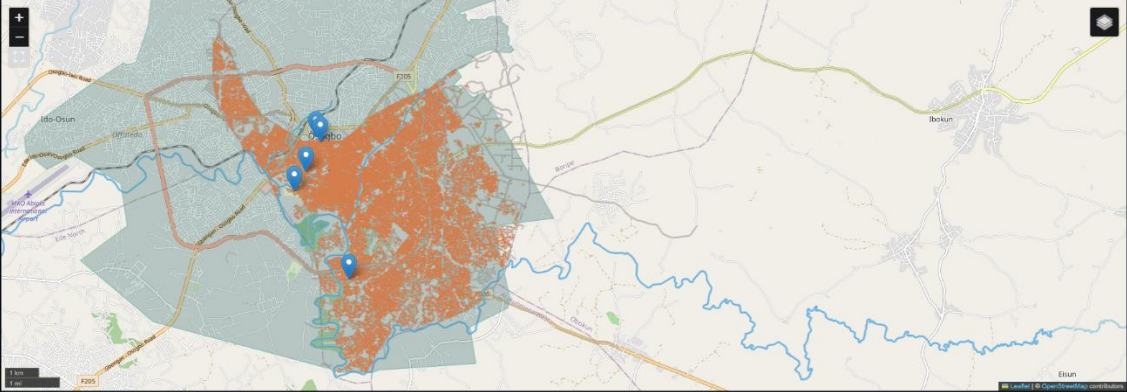
Samples/Screenshot of tested LGAs



Ibadan North LGA

Preview map

Every extracted layer is shown together below. Use the layer control (top-right of the map) to toggle individual layers on or off, useful since dense layers like buildings can visually overwhelm the map when everything is shown at once.



Download results

[Download Osogbo OSM data \(.zip\)](#)

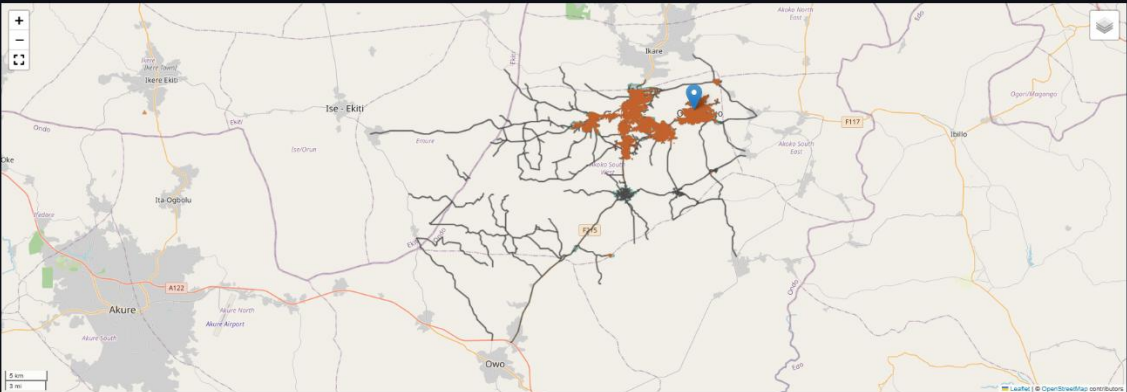
[Manage app](#)

Osogbo LGA

[Warnings](#)

Preview map

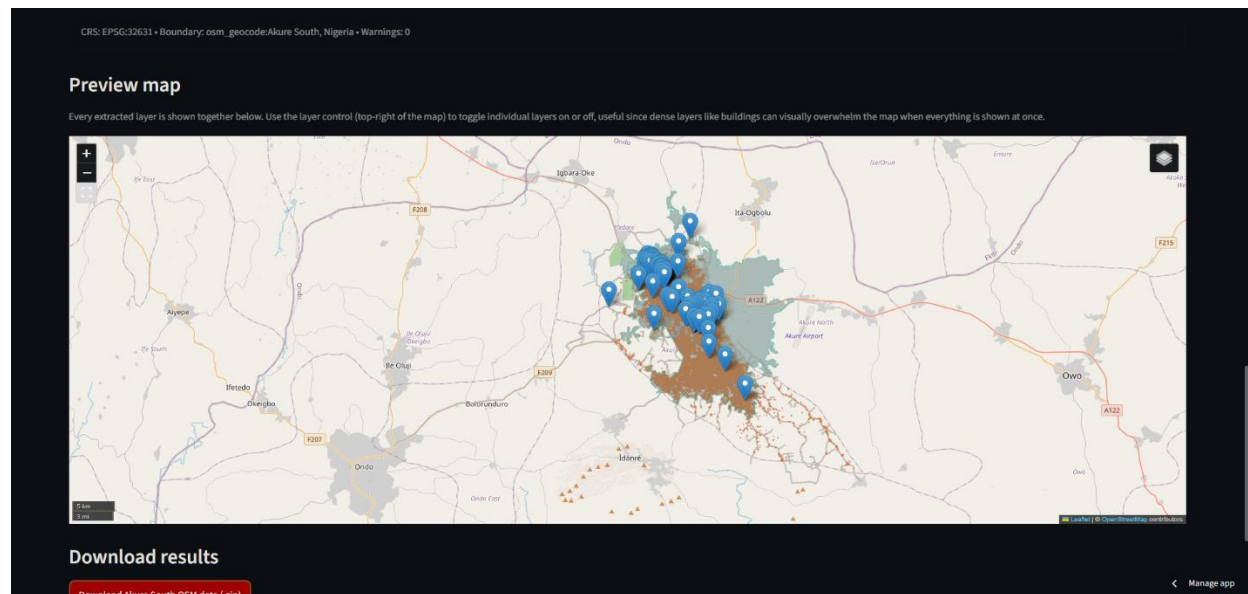
Every extracted layer is shown together below. Use the layer control (top-right of the map) to toggle individual layers on or off, useful since dense layers like buildings can visually overwhelm the map when everything is shown at once.



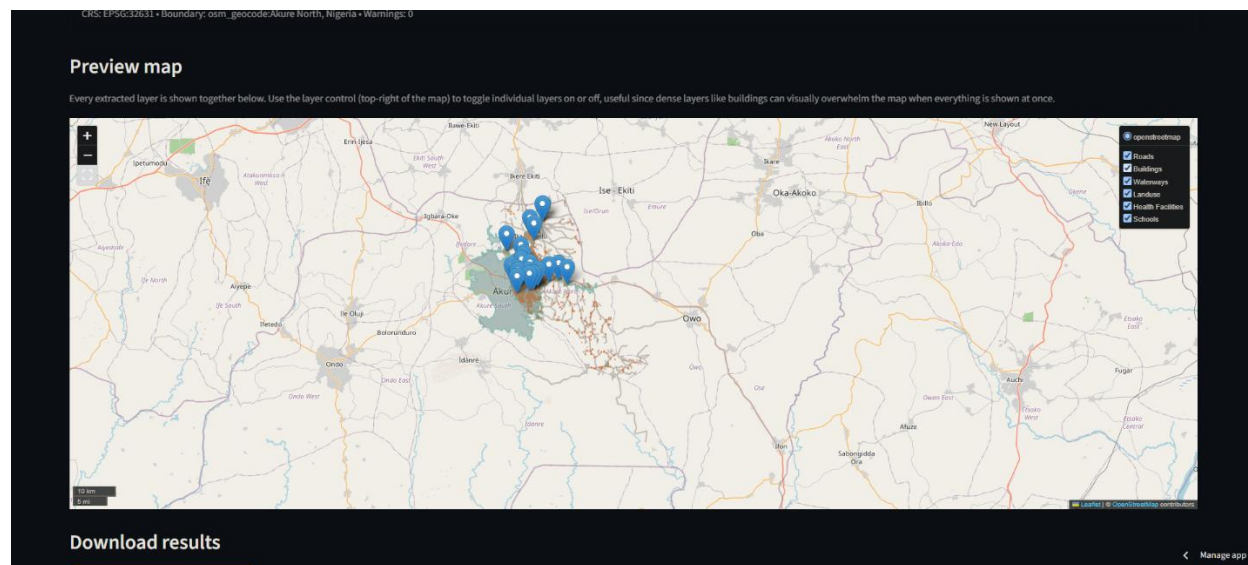
Download results

[Manage app](#)

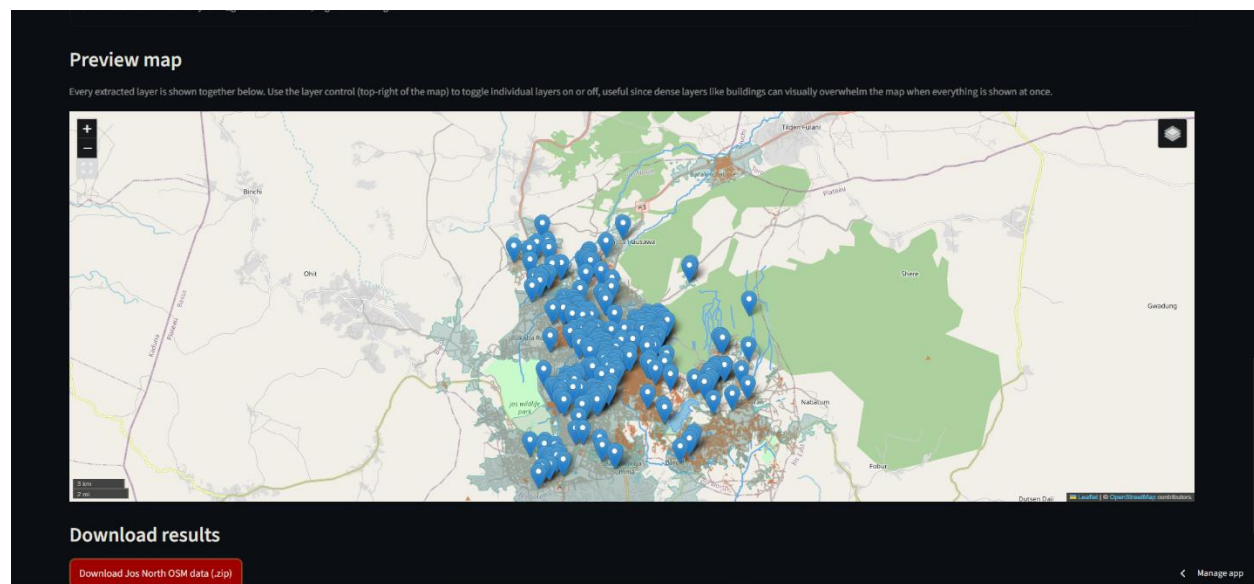
Akoko South West



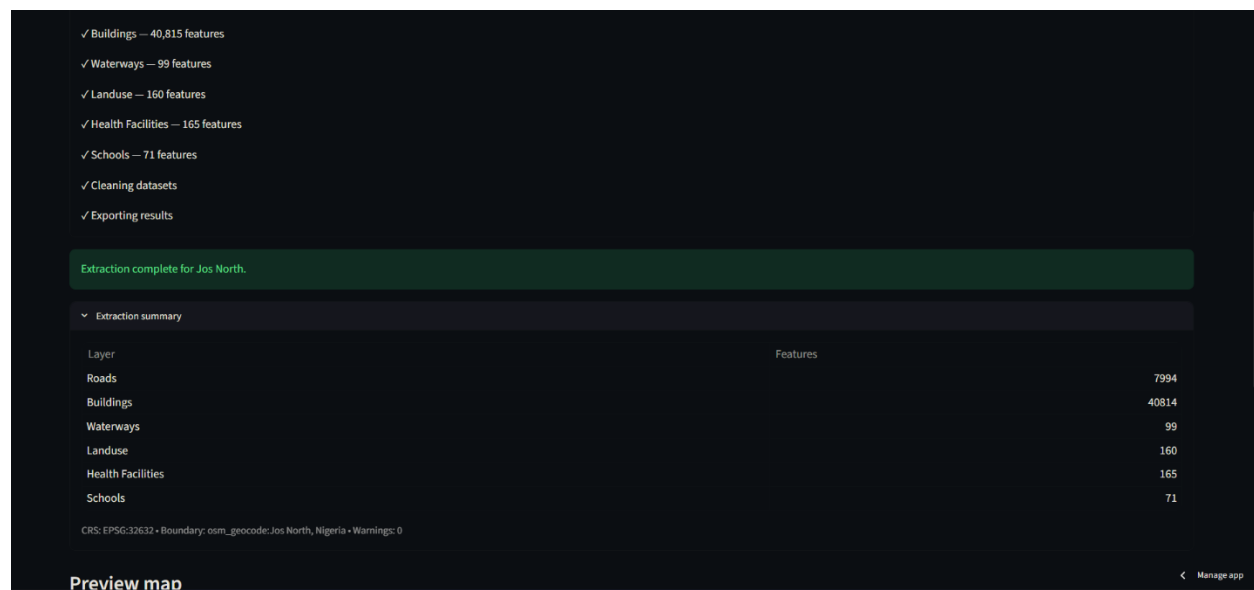
Akure South



Akure North



Jos North



Screenshot showing list of extracted layer by the tool